

Datenstrukturen

Programmieren und Software-Engineering Theorie

25. März 2026

Grundbegriffe

- Einfachster Datentyp: Variable
- Komplexere Datentypen: Klassen und Objekte
- Organisation mehrerer Elemente: Datenstrukturen

Grundbegriffe

- Lineare Datenstrukturen
 - Array
 - Lists
 - Stack
 - Queue
- Dictionaries / Maps
- Sets (Mengen)
- Graphenbasierte Datenstrukturen:
 - Bäume
 - allgemeine Graphen

Array

- Zusammenhängende Anordnung mehrerer Elemente im Speicher
- Zugriff auf bestimmte Position mit Index $i = 0, 1, 2, \dots$
- Bei vielen Sprachen existiert Array als elementarer Datentyp
- Operationen wie *Einfügen* oder *Löschen* werden dann in höheren Datentypen umgesetzt.
- Beispiel Java: `Array []` versus `ArrayList`

Array

Array mit 10 Elementen (Index unter den Zellen).

7	3	12	5	9	1	4	0	0	0
0	1	2	3	4	5	6	7	8	9

Array

Array nach dem Einfügen eines Elements 7 an der Stelle mit Index 4. Die Elemente 9, 14, und 4 wurden um eine Position nach rechts verschoben.

7	3	12	5	7	9	14	4	0	0
0	1	2	3	4	5	6	7	8	9

Array

Nun soll das Element an der Stelle mit Index 2 gelöscht werden.

7	3	12	5	7	9	14	4	0	0
0	1	2	3	4	5	6	7	8	9

Array

Die Elemente 12, 5, 7, 9, 14 und 4 wurden um eine Position nach links verschoben.

7	3	5	7	9	14	4	0	0	0
0	1	2	3	4	5	6	7	8	9

Array (Analyse)

Laufzeit der wichtigsten Operationen:

- Zugriff mit Index: $O(1)$
- Einfügen: $O(n)$
- Entfernen: $O(n)$

Array (Java)

Interne Umsetzung von ArrayList in Java

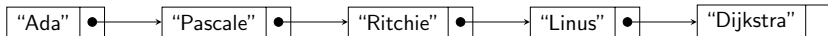
- Arrays existieren in Java als elementare Datenstruktur.
- `ArrayList<E>` bietet einen komfortableren Umgang mit Arrays an.
- `ArrayList<E>` nutzt intern ein `Object[]`.
- Beim Erweitern wird bei Bedarf ein größeres Array alloziert und die Elemente kopiert.
- Indexzugriff $O(1)$, Einfügen in der Mitte $O(n)$ wegen Verschieben der nachfolgenden Elemente.

Operationen und ihre Komplexität

- `add(e)` → Am Ende einfügen, amortisiert $O(1)$.
- `add(i, e)` → An Position `i` einfügen, $O(n)$.
- `get(i)` → Zugriff per Index, $O(1)$.
- `remove(i)` → $O(n)$ im Worst Case.

Verkettete Listen

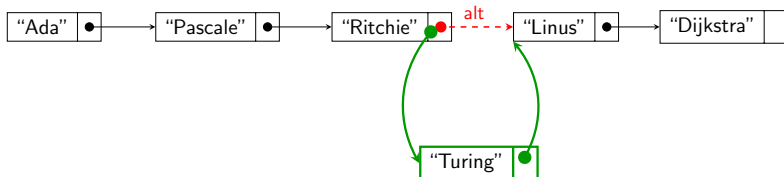
- Elemente liegen verteilt im Speicher
- Werden miteinander mittels Zeiger/Pointer (=Objektreferenzen) verkettet
- Einfach- vs. doppelt verketteter Liste



Verkettete Listen: Einfügen

Einfügen von **"Turing"** zwischen "Ritchie" und "Linus":

- 1 Neuen Knoten erzeugen, seinen Zeiger auf "Linus" setzen
- 2 Zeiger von "Ritchie" auf den neuen Knoten abändern



Verkettete Listen: Löschen

Löschen von "Pascale" aus der Liste:

- 1 Zeiger von "Ada" auf Nachfolger von "Pascale" (= "Ritchie") setzen
- 2 Knoten "Pascale" freigeben



Verkettete Listen (Analyse)

Laufzeit der wichtigsten Operationen:

- Zugriff Index: $O(n)$
- Einfügen an beliebiger Stelle: $O(n)$
- Löschen: $O(n)$
- Zugriff/Einfügen/Löschen mit Iterator: $O(1)$

Verkettete Listen (Java)

- Java stellt mit `LinkedList<E>` eine doppelt verkettete Liste bereit.
- `LinkedList`: Einfügen/Entfernen am Anfang/Ende und über Iterator $O(1)$, zufälliger Indexzugriff $O(n)$.
- Für Random Access ist `ArrayList` die bessere Wahl ($O(1)$ per Index).

Stack

Ein **Stack** folgt dem Prinzip **Last In — First Out (LIFO)**: Das zuletzt eingefügte Element wird als erstes wieder entfernt. Typische Operationen sind:

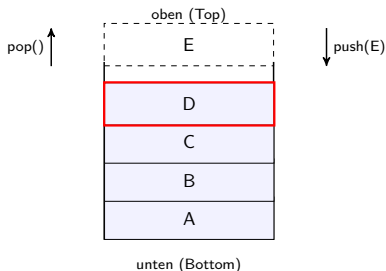
- **push**: Einfügen eines Elements oben auf den Stack.
- **pop**: Entfernen des obersten Elements.
- **peek/top**: Zugriff auf das oberste Element, ohne es zu löschen.



Stacks lassen sich einfach mit Arrays oder verketteten Listen realisieren und werden z. B. bei Funktionsaufrufen (Call Stack) oder in Undo-Mechanismen eingesetzt.

Stack

Schematischer Stack (LIFO): Nur das oberste Element (rot markiert) kann entfernt (pop) werden, bzw. kann auch nur oben hinzugefügt werden (push).

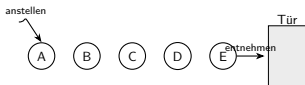


Queue

Eine **Queue** (Warteschlange) folgt dem Prinzip **First In – First Out (FIFO)**: Das zuerst eingefügte Element wird auch als erstes wieder entfernt. Typische Operationen sind:

- enqueue: Einfügen eines Elements am Ende der Queue.
- dequeue: Entfernen des ersten Elements.
- front: Zugriff auf das erste Element, ohne es zu löschen.

Queues werden oft in Betriebssystemen (Prozessplanung), Netzwerken (Datenpaketverwaltung) oder Druckersystemen verwendet.



Schematische Warteschlange (Queue): vorne (rechts) wird entnommen, hinten (links) wird angestellt.

Stack/Queue (Java)

In Java lassen sich Stack/Queue effizient mit der Deque-Schnittstelle umsetzen (z.B. ArrayDeque).

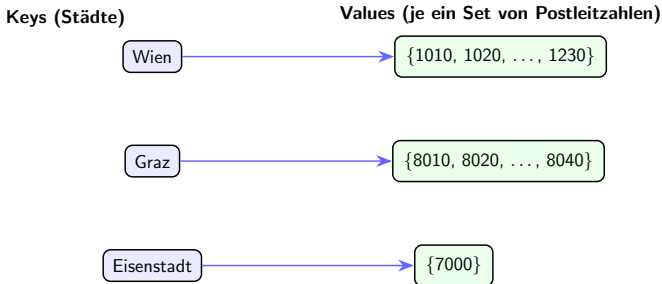
- `push()/pop()` bzw. `addFirst()/removeFirst()` für Stack-Verhalten.
- `add()/poll()` oder `addLast()/removeFirst()` für Queue-Verhalten.
- `ArrayDeque` ist in der Regel schneller als `LinkedList` für diese Einsatzzwecke und liefert $O(1)$ -Operationen an beiden Enden.
- Funktionsweise:
 - Es gibt zwei Zeiger:
 - `head`: zeigt auf den ersten gültigen Slot
 - `tail`: zeigt auf den nächsten freien Slot am Ende
 - Das Array wird ringförmig genutzt
 - Wenn `tail` ans Ende des Arrays kommt, springt es wieder auf Index 0.
 - Wenn `head` vor Index 0 geht, springt es ans Ende.

Dictionaries/Maps

- Dictionaries oder Maps **Schlüssel – Wert**-Paare.
- Oft möchte man gespeicherte Objekte *schnell* aufgrund eines *Schlüssels* (id, name, ...) finden, bzw. darauf zugreifen.
- Beispiel:
 - Finde Mitarbeiter mit Personalnummer
 - Finde Immobilie mittels Anzeigennummer
- Das Suchen in Arrays, ArrayLists etc. ist aufwändig, da die Datenstruktur zunächst durchsucht werden muss.
- **Ziel:** Auffinden eines Elementes aufgrund des **Schlüssels (Key)** in (quasi) konstanter Zeit $O(1)$, d.h. ohne viele Elemente in der Datenstruktur betrachten zu müssen.

Dictionaries/Maps

Beispiel: Es werden Städten (=Schlüssel) die Mengen von Postleitzahlen (=Werte) zugeordnet. Der Wert ist hier also eine Menge (Datenstruktur Set).



Hashfunktionen und Hashtabellen

- Mittels *Hashfunktionen* wird zu einem Schlüssel ein Hashwert berechnet, der als Index in einer *Hashtabelle* dient.
- Schlüssel (z.B. Strings) sollen schnell auf Speicherplätze (Buckets) einer Tabelle abgebildet werden.
- Dazu wird eine *Hashfunktion* h verwendet, die einen Schlüssel k auf einen ganzzahligen Index $h(k) \in \{0, \dots, m-1\}$ (Tabellengröße m) projiziert.
- Hashtabellen verbinden eine (große) Array-Struktur fester Länge m mit einer Hashfunktion. Jeder Eintrag (Bucket) kann (abhängig vom Verfahren) direkt ein Element oder eine kleine Sekundärstruktur (Liste/Baum) enthalten.
- Ziel: Erwartete $O(1)$ Zeit für Suchen, Einfügen und Löschen.

Hashfunktion

- Deterministisch: Gleicher Schlüssel $\Rightarrow$ gleicher Hashwert.
- Effizient: Berechnung in (amortisiert) $O(1)$ und sehr schnell (wenige CPU-Operationen).
- Gleichmäßige Verteilung: Schlüssel sollen möglichst gleich über $\{0, \dots, m-1\}$ verteilt werden (minimiert Kollisionen).

Hashfunktion

$$h(k) = h'(k) \bmod m$$

- $h'(k)$: Hashwertfunktion (Java: `hashCode()`). Ordnet einem Schlüssel k einen großen Integer zu.
- Hashfunktion $h(k)$ (Berechnung des Bucketindex): faltet den großen Integer $h'(k)$ mittels Modulo-Operation in den gültigen Indexbereich der Tabelle mit Größe m .

Grundidee einer Hashtabelle

"Katze" $\longrightarrow h' = 11 \longrightarrow 11 \bmod 8 = 3$

"Hund" $\longrightarrow h' = 26 \longrightarrow 26 \bmod 8 = 2$

0	1	2	3	4	5	6	7
		Hund	Katze				Maus

"Maus" $\longrightarrow h' = 7 \longrightarrow 7 \bmod 8 = 7$

Grundidee einer Hashtabelle (Kollision)

"Katze" $\longrightarrow h' = 11 \longrightarrow 11 \bmod 8 = 3$

"Hund" $\longrightarrow h' = 26 \longrightarrow 26 \bmod 8 = 2$

0	1	2	3	4	5	6	7
		Hund	Katze				Maus

"Hase" $\longrightarrow h' = 19 \longrightarrow 19 \bmod 8 = 3$

"Maus" $\longrightarrow h' = 7 \longrightarrow 7 \bmod 8 = 7$

Hashtabellen: Grundbegriffe

- *Kollision*: Zwei verschiedene Schlüssel $k_1 \neq k_2$ mit $h(k_1) = h(k_2)$.
- *Lastfaktor* (load factor) $\alpha = \frac{n}{m}$ mit n = Anzahl gespeicherter Elemente.
- *Rehashing*: Vergrößern der Tabelle (typisch Faktor 2) und Neuverteilen aller Elemente, sobald α einen Schwellwert überschreitet.

Beispiele für Hashwertfunktionen

- Strings: Polynomielles Rolling-Hash
$$h'(s_0s_1 \dots s_{L-1}) = \sum_{i=0}^{L-1} s_i \cdot B^{L-1-i}$$
 mit Basis B (z.B. 131, 257)
und ggf. Reduktion mit großem Prim P vor Modulo m .
- Integer: Mischung (Bit-Mixing), z.B. Multiplikation mit großer ungerader Konstante und Bit-Shifts um Probleme mit Verteilung zu vermeiden.

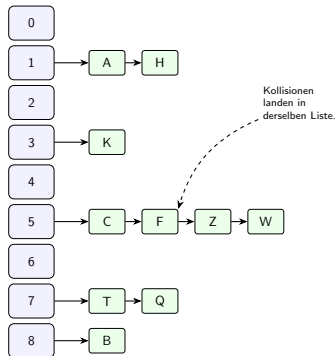
Kollisionsbehandlung (Überblick)

Es gibt zwei voneinander unabhängige Ansätze zur Behandlung von Kollisionen:

- *Getrennte Verkettung* (Chaining): Jeder Bucket hält eine (kurze) Liste / Baum der kollidierenden Elemente.
- *Offene Adressierung* (Sondieren): Bei Kollisionen wird versucht das Element an einer anderen Stelle im Array abzulegen (Sondieren).

Methode 1: Verkettung der Überläufer

Hashtabelle mit Verkettung (Chaining): Jeder **Bucket** zeigt (falls nicht leer) auf den Kopf einer Liste kollidierender **Elemente**. Die erwartete Listenlänge entspricht dem Lastfaktor $\alpha = n/m$.



Erwartete Länge einer Liste: $\alpha = n/m$ (Lastfaktor) bei guter Hashfunktion $\Rightarrow$ erwartete Kosten $O(1 + \alpha)$.

Methode 2: Offene Hashverfahren (Sondieren)

- Alternative Strategie für Kollisionsbehandlung
- Wenn Platz belegt, berechne neuen Platz mittels Sondierfunktion $f(i)$
- Dabei: erster Versuch: $i = 0$, zweiter Versuch: $i = 1$, usw.

Bei belegtem Platz wird eine neuer Platz wie folgt berechnet

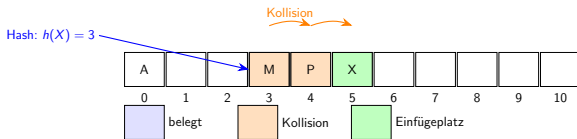
$$h_i(k) = (h(k) + f(i)) \bmod m, \quad i = 0, 1, 2, \dots$$

- Lineares Sondieren: $f(i) = i$
- Quadratisches Sondieren: $f(i) = i + i^2 *$

* Allgemein auch $f(i) = c_1 i + c_2 i^2$ mit Konstanten c_1, c_2

Illustration: lineares Sondieren

Einfügen von Schlüssel X mit Hash 3: Position 3 ist belegt, lineares Sondieren prüft 4 (auch belegt), findet 5 frei und legt dort ab. Die Anzahl besuchter Buckets hängt vom Cluster vor der Zielposition ab; geringe Auslastung minimiert durchschnittliche Sondenlänge.



Methode 3: Double Hashing

Eine weitere Möglichkeit ist Double Hashing: $h_i(k) = h_1(k) + i \cdot h_2(k)$. Hierbei werden zwei verschiedene Hashfunktionen h_1 und h_2 verwendet, um die Sondierfolge zu bestimmen. Dies kann helfen, Kollisionen weiter zu reduzieren und eine bessere Verteilung der Elemente in der Hashtabelle zu erreichen. Die Wahl von h_2 ist hierbei jedoch kritisch, ebenso ist das Verfahren aufwändiger als lineares oder quadratisches Sondieren.

Vor- und Nachteile der Verfahren:

- Chaining: Einfach, Performance robust, benötigt Zeiger/Overhead.
- Offene Adressierung: Cache-freundlich (alles im Array), aber sensibler gegenüber hoher Auslastung; Clusterbildung möglich.

Beispiel: lineares Sondieren

- Gegeben sei eine Hashtabelle der Größe $m = 8$ mit der Hashwertfunktion $h'(k) = k$.
- Zu verwenden ist lineares Sondieren, also $f(i) = i + i^2$. Somit $h_i(k) = (h'(k) + i) \bmod 8$.
- Einzufügende Schlüssel (in dieser Reihenfolge): 10, 19, 31, 22, 14, 16

Schrittweises Einfügen (lineares Sondieren)

- 10: $h_0(10) = 10 \bmod 8 = 2 \Rightarrow$ Slot 2 frei $\Rightarrow$ einfügen.
- 19: $h_0(19) = 19 \bmod 8 = 3 \Rightarrow$ Slot 3 frei $\Rightarrow$ einfügen.
- 31: $h_0(31) = 31 \bmod 8 = 7 \Rightarrow$ Slot 7 frei $\Rightarrow$ einfügen.
- 22: $h_0(22) = 22 \bmod 8 = 6 \Rightarrow$ Slot 6 frei $\Rightarrow$ einfügen.
- 14: $h_0(14) = 14 \bmod 8 = 6 \Rightarrow$ Kollision bei 6 (22)
 $h_1(14) = (14 + 1) \bmod 8 = 7$ (31, belegt)
 $h_2(14) = (14 + 2) \bmod 8 = 0$ (frei) $\Rightarrow$ Slot 0.
- 16: $h_0(16) = 16 \bmod 8 = 0$ Kollision bei 0 (14)
 $h_1(16) = (16 + 1) \bmod 8 = 1$ (frei) $\Rightarrow$ Slot 1.

Endzustand der Hashtabelle

Index	0	1	2	3	4	5	6	7
Wert	14	16	10	19			22	31

Beispiel: lineares Sondieren

Beobachtung: Die Clusterbildung zeigt sich daran, dass die Kollision von 14 (Start bei 6) die Sondenfolge bis zum Anfang (Wrap-Around) verlängert; 16 trifft danach sofort erneut auf eine belegte Zelle. Längere zusammenhängende belegte Abschnitte wachsen schneller (Primär-Cluster).

Beispiel: quadratisches Sondieren

- Gegeben sei eine Hashtabelle der Größe $m = 8$ mit der Hashwertfunktion $h'(k) = k$.
- Zu verwenden ist quadratisches Sondieren mit der Sondierfunktion $f(i) = i + i^2$. Somit $h_i(k) = (h'(k) + i + i^2) \bmod 8$.
- Einzufügende Schlüssel (in dieser Reihenfolge): 10, 19, 31, 22, 14, 16

Schrittweises Einfügen (quadratisches Sondieren)

- 10: $h_0(10) = 10 \bmod 8 = 2$ frei $\Rightarrow$ Slot 2.
- 19: $h_0(19) = 19 \bmod 8 = 3$ frei $\Rightarrow$ Slot 3.
- 31: $h_0(31) = 31 \bmod 8 = 7$ frei $\Rightarrow$ Slot 7.
- 22: $h_0(22) = 22 \bmod 8 = 6$ frei $\Rightarrow$ Slot 6.
- 14: $h_0(14) = 14 \bmod 8 = 6$ belegt (22).
 $h_1(14) = (14 + 1 + 1) \bmod 8 = 0$ frei $\Rightarrow$ Slot 0.
- 16: $h_0(16) = 16 \bmod 8 = 0$ belegt (14).
 $h_1(16) = (16 + 1 + 1) \bmod 8 = 2$ belegt (10);
 $h_2(16) = (16 + 2 + 4) \bmod 8 = 6$ belegt (22);
 $h_3(16) = (16 + 3 + 9) \bmod 8 = 4$ frei $\Rightarrow$ Slot 4.

Endzustand der Hashtabelle

Index	0	1	2	3	4	5	6	7
Wert	14		10	19	16		22	31

Beispiel: quadratisches Sondieren

Beobachtung: Kollisionen (z.B. Startindex 6 für 22 und 14) führen zu Sprüngen, die kein zusammenhängendes primäres Cluster erzeugen (Slots: $6 \rightarrow 0$). Dennoch teilen alle Schlüssel mit gleichem Start-Hash dieselbe Sondenfolge (sekundäre Cluster). Mit $m = 8$ (Potenz von 2) muss geeignete Wahl von c_1, c_2 getroffen werden, um genügend unterschiedliche Slots zu erreichen.

Auslastung

- Verkettung: Gute Performance auch bei hoher Auslastung (z.B. $\alpha > 1$).
- Offene Adressierung: Performance verschlechtert sich stark bei $\alpha > 0.7$ (lineares Sondieren) bzw. $\alpha > 0.9$ (quadratisches Sondieren).
- Bei offener Adressierung muss gelten $\alpha < 1$ (sonst kein Platz mehr für neue Elemente).

Java

- HashMap verwendet Verkettung (Chaining) mit dynamischen Buckets (z.B. LinkedList oder TreeMap bei vielen Kollisionen).
- Objekte sind als Schlüssel in HashMap nutzbar, wenn sie `hashCode()` und `equals()` konsistent implementieren.
- Eingebaute unveränderliche Typen (`String`, `Integer`, `Long`, ...) bieten diese Funktionalität bereits an.
- Generell für unveränderliche (*immutable*) Objekte sinnvoll, da sich Hashwert nicht ändert.

Zusammenfassung und Analyse

Laufzeitanalyse von Dictionaries:

- Erfolgreiche Suche: amortisiert $O(1)$
- Einfügen: amortisiert $O(1)$
- Löschen: amortisiert $O(1)$
- Worst Case (pathologisch viele Kollisionen): $O(n)$

Konsistenzanforderungen:

- Hash und Gleichheit müssen konsistent sein: $k_1 = k_2 \Rightarrow h(k_1) = h(k_2)$.
- Schlüssel dürfen sich nach Einfügen nicht (logisch) ändern.

Zusammenfassung:

Eine gute Hashfunktion ist schnell, verteilt Schlüssel gleichmäßig und minimiert so Kollisionen. Kollisionsbehandlung (Chaining oder offene Adressierung) plus kontrollierter Lastfaktor sichern amortisierte $O(1)$ -Operationen für Insert, Lookup und Delete.

Java-Maps

Java verwendet HashMap (bzw. LinkedHashMap, TreeMap):

```
// Erzeugen
Map<String,Object> d = new HashMap<>();           // leeres Map
Map<String,Object> person = new HashMap<>();
person.put("name", "Ada");
person.put("age", 37);

// Zugriff / Einfügen / Überschreiben
person.put("city", "London");                     // Einfügen
person.put("age", 38);                             // Überschreiben
System.out.println(person.get("name"));           // Direktzugriff
```

Java-Maps

```
// Sicherer Zugriff
System.out.println(person.get("email"));           // -> null
System.out.println(person.getDefault("email", "kein Eintrag"));

// Existenztest
if (person.containsKey("age")) { ... }

// Entfernen
person.remove("city");                             // entfernt, kein Fehler
                                                    // bei fehlendem Key
Integer age = (Integer) person.remove("age"); // entfernt + liefert Wert

// Iteration
for (String k : person.keySet()) { ... }

for (Map.Entry<String, Object> e : person.entrySet()) {
    String k = e.getKey(); Object v = e.getValue();
}

for (Object v : person.values()) { ... }
```

Sets

- Ein *Set* entspricht einer mathematischen Menge.
- Gleiche Elemente können in einem Set nur einmal enthalten sein.
- In einem Set kann man (im Gegensatz zur `ArrayList`) nicht mittels Index auf bestimmte Elemente zugreifen.
- Operationen auf Sets:
 - Hinzufügen eines Elementes
 - Entfernen eines Elementes
 - Überprüfen ob ein Element im Set enthalten ist

Anmerkung: Die konkrete Umsetzung der oben definierten Anforderungen an die Datenstruktur Set kann auf verschiedene Arten erfolgen. Eine sehr gebräuchliche und effiziente Methode ist die Verwendung von Hashtabellen aber auch baum-basierte Sets sind möglich.

Sets mit Hashtabelle

- Jedes Element wird durch seinen Hashwert in eine bestimmte Position (Bucket) der Hashtabelle eingefügt.
- Dies ermöglicht einen schnellen Zugriff auf die Elemente, da die Suche, das Hinzufügen und das Entfernen im Durchschnitt in konstanter Zeit $O(1)$ erfolgen können.
- Vergleiche mit naiver Implementierung als Liste: jeder Zugriff $O(n)$ Zeit

Datentyp Set (Standardimplementierung: HashSet)

Wichtige Operationen:

- `Set<Integer> s = new HashSet<>();` → Erzeugung
- `s.add(x);` → fügt ein Element hinzu (tut nichts, wenn schon vorhanden).
- `s.remove(x);` → entfernt ein Element (gibt boolean zurück).
- `s.clear();` → leert das Set.
- Iteration: `for (T e : s) ....`

Eigenschaften:

Die Elemente sind im Set einzigartig. `HashSet` garantiert keine Reihenfolge; Zugriff/Einfügen/Entfernen benötigt im Durchschnitt $O(1)$.

Set mittels Bäumen

- Umsetzung des *Set*-Konzeptes mittels *Bäumen*.
- Elemente können in $O(\log n)$ gesucht, eingefügt und gelöscht werden.
- Zusatznutzen: die Elemente sind immer sortiert!
- Zugriff auf nächst- größeres/kleineres Element schnell möglich

In Java steht mit `TreeSet` eine standardisierte Baum-basierte Set-Implementierung zur Verfügung, das die Elemente sortiert hält ($O(\log n)$ Operationen).

Laufzeitanalyse

Laufzeitanalyse verschiedener Operationen bei Größe n der Datenstruktur:

Operation	Array(List)	Verkettete Liste	Hash-Set	Tree-Set
add (end)	$O(1)$	$O(1)$	$O(1)$	$O(\lg n)$
remove	$O(n)$	$O(n)$	$O(1)$	$O(\lg n)$
remove (iterator)	$O(n)$	$O(1)$	$O(1)$	$O(\lg n)$
get (index)	$O(1)$	$O(n)$	$O(1)$	$O(\lg n)$
access (position)	$O(1)$	$O(n)$		
find (element)	$O(n)$	$O(n)$	$O(1)$	$O(\lg n)$
insert (position)	$O(n)$	$O(n)$		
insert (iterator)	$O(n)$	$O(1)$		

Laufzeitanalyse

- In der Tabelle scheint das Hash-Set dem Tree-Set eindeutig überlegen zu sein.
- Das Tree-Set bietet jedoch den Vorteil, dass die enthaltenen Elemente effizient sortiert ausgegeben werden können, und es benötigt wesentlich weniger zusätzlichen Speicher als das Hash-Set.
- Außerdem ist bei Hashtabelle eine Reorganisation/Neuaufbau notwendig, wenn Auslastung zu groß wird. Aufwand $O(n)$, amortisiert $O(1)$.